

Propuesta de Trabajos de Ampliación: Módulo Acceso a Datos (2º DAM)

Esta propuesta está diseñada para profundizar en tecnologías de persistencia avanzada, optimización y despliegue moderno utilizando Python como lenguaje vehicular.

Trabajo 1: Dockerización de Ecosistemas de Datos

Objetivo: Eliminar el "en mi ordenador funciona" mediante el uso de contenedores.

- **Contenido Teórico:**
 - ¿Qué es un contenedor frente a una máquina virtual?
 - Anatomía de un archivo docker-compose.yml.
 - Persistencia en Docker: volúmenes y redes.
- **Parte Práctica:**
 - Crear un entorno con docker-compose que levante simultáneamente una base de datos PostgreSQL y un Adminer (interfaz web de gestión).
 - Configurar un script Python que detecte automáticamente si la base de datos está lista y cree las tablas necesarias.
- **Recursos recomendados:** Docker Desktop, imágenes oficiales de Docker Hub.

Trabajo 2: High Performance - Pooling de Conexiones

Objetivo: Entender que el patrón Singleton tiene límites en entornos multiusuario.

- **Contenido Teórico:**
 - El coste computacional de abrir/cerrar una conexión TCP/IP.
 - Diferencia entre una conexión única (Singleton) y un Pool de conexiones.
 - Estrategias de gestión: conexiones mínimas, máximas y tiempos de espera.
- **Parte Práctica:**
 - Implementar un ejemplo usando psycopg2.pool o el pooling nativo de SQLAlchemy.
 - Realizar un test de estrés: comparar el tiempo que tarda el sistema en realizar 200 inserciones rápidas abriendo una conexión nueva cada vez vs. usando un Pool.
- **Recursos recomendados:** Librería psycopg2.pool o SQLAlchemy.

Trabajo 3: PostgreSQL Avanzado - El modelo Objeto-Relacional

Objetivo: Explotar PostgreSQL más allá de las tablas simples, usando tipos complejos.

- **Contenido Teórico:**
 - Tipos de datos avanzados: ARRAY y Composite Types (tipos definidos por el usuario).
 - Búsquedas avanzadas sobre JSONB (índices GIN).
 - Herencia de tablas en PostgreSQL (conceptos básicos).
- **Parte Práctica:**
 - Crear una pequeña base de datos de catálogo de películas, videojuegos o libros. La tabla principal deberá usar un ARRAY para guardar géneros o etiquetas, por ejemplo acción, aventura o comedia; un campo JSONB para almacenar datos adicionales que cambian según el tipo de elemento; y un tipo compuesto para agrupar información relacionada, por ejemplo una valoración formada por puntuación, fecha y comentario breve.

- Aclaración: por datos adicionales que cambian según el tipo de elemento se entiende información específica que no todos los registros tienen por igual. La idea es que se vea que JSONB permite guardar información flexible sin crear una columna diferente para cada posible característica.
- Ejemplo: una película puede tener duración, director e idioma original; un videojuego puede tener plataforma, modo online y número de jugadores; y un libro puede tener autor, editorial, ISBN y número de páginas.
- Ejemplo de JSONB para un videojuego: { "plataforma": "PS5", "modo_online": true, "pegi": 12, "numero_jugadores": 4 }.
- Demostrar consultas SQL sencillas que filtren elementos por una etiqueta del ARRAY, consulten un valor concreto dentro del JSONB y muestren los campos del tipo compuesto. Como ampliación opcional, se puede crear una tabla base recurso y una tabla heredada, por ejemplo pelicula o videojuego, para ilustrar la herencia de tablas en PostgreSQL.
- **Recursos recomendados:** Documentación oficial de PostgreSQL (sección Object-Relational).

Trabajo 4: Persistencia en Tiempo Real - Redis como Caché

Objetivo: Introducir las bases de datos NoSQL Key-Value para datos volátiles de alta velocidad.

- **Contenido Teórico:**
 - Bases de datos en memoria vs. bases de datos en disco.
 - Casos de uso: caché de sesiones, leaderboards y logs de corta duración.
 - Estructuras de datos en Redis: Strings, Lists y Hashes.
- **Parte Práctica:**
 - Simular un sistema de últimos usuarios conectados para la app de biometría.
 - En lugar de escribir en Postgres cada vez que alguien abre la cámara, que puede ser lento, los intentos se guardan en Redis con un tiempo de expiración (TTL) de 5 minutos.
- **Recursos recomendados:** Librería redis-py, imagen de Docker de Redis.

Trabajo 5: Bases de Datos Vectoriales - Búsqueda por Similitud

Objetivo: Introducir el concepto de base de datos vectorial y entender cómo permite buscar información por similitud semántica, no solo por coincidencia exacta de palabras.

- **Contenido Teórico:**
- Qué es un embedding: representación numérica de un texto, imagen u otro dato en forma de vector.
- Diferencia entre búsqueda tradicional por palabras clave y búsqueda semántica por similitud.
- Conceptos básicos de distancia o similitud entre vectores: similitud coseno, distancia euclídea y top k resultados.
- Casos de uso: buscadores inteligentes, recomendadores, chatbots con documentos propios y recuperación de información.
- **Parte Práctica:**
- Crear un mini buscador semántico con una colección pequeña de datos, por ejemplo 10 películas, 10 libros, 10 productos o 10 incidencias técnicas. Cada elemento tendrá un título, una descripción breve y alguna etiqueta o metadato sencillo.
- Guardar esos textos en una base vectorial sencilla, por ejemplo ChromaDB, FAISS o PostgreSQL con la extensión pgvector. Para simplificar, se puede usar una librería que genere los embeddings automáticamente.

- Realizar varias búsquedas en lenguaje natural y mostrar los 3 resultados más parecidos. Ejemplo: buscar "películas de aventuras para niños" y obtener como resultado elementos relacionados aunque no contengan exactamente esas mismas palabras.
- Comparar brevemente el resultado con una búsqueda tradicional por palabra clave para observar la diferencia entre coincidencia literal y similitud semántica.
- Entregable práctico: script Python sencillo, base de datos o colección vectorial creada, ejemplos de consultas y una breve explicación del funcionamiento.
- **Recursos recomendados:** ChromaDB, FAISS o pgvector; librerías sentence-transformers o embeddings locales sencillos; documentación oficial de las herramientas utilizadas.

Sugerencia de Evaluación y Tiempos

La presentación en clase y la preparación de diapositivas se sustituyen por la grabación de un vídeo explicativo. El vídeo deberá explicar los conceptos principales del trabajo y acompañarse de una muestra práctica del funcionamiento.

Actividad	Tiempo estimado	Entregable
Investigación y lectura	1,5 horas	Guion o esquema del vídeo explicativo.
Desarrollo del código de ejemplo	2,5 horas	Repositorio GitHub / carpeta de código.
Grabación del vídeo explicativo	1 hora	Vídeo con explicación de los conceptos y muestra práctica del funcionamiento.